

# Software Design

## Internal Design

The repository is structured around a small set of explicit interfaces and adapter classes.

| Area                    | Primary files                                                                                                                                  | Design role                                                          |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Game contract           | <code>ConsoleTest/Games/IGame.cs</code>                                                                                                        | Defines the lifecycle and state methods used by all games.           |
| Game implementations    | <code>ConsoleTest/Games/Snake.cs</code> , <code>Tetris.cs</code> , <code>Education.cs</code>                                                   | Contain the rule sets, scoring, and drawing behavior.                |
| Main application        | <code>ConsoleTest/Program.cs</code>                                                                                                            | Orchestrates config, state transitions, rendering, and score export. |
| Display abstraction     | <code>PixelBoardDisplay/IDisplay.cs</code> , <code>DisplayHelper.cs</code>                                                                     | Normalizes board drawing and LCD-style output.                       |
| Display implementations | <code>PixelBoardDisplay/ConsoleDisplay.cs</code> , <code>ArduinoDisplay.cs</code>                                                              | Render the same game state to different targets.                     |
| API clients             | <code>ConsoleTest/Leaderboardclient.cs</code> , <code>PixelCatsClient/ApiClient.cs</code> , <code>PixelCatsClient/PixelCatsApiClient.cs</code> | Encapsulate external HTTP calls.                                     |
| File watchers           | <code>PixelCatsClient/ProgramHelpers.cs</code> , <code>ScoreWatcher.cs</code>                                                                  | Read and react to score-file updates.                                |
| Tests                   | <code>ConsoleTest.Tests/</code>                                                                                                                | Validate the main behavioral contracts.                              |

## Major Modules and Responsibilities

### `ConsoleTest/Program.cs`

Owens the application loop, game selection, game-over transitions, score export, and leaderboard submission. It also loads configuration from `appsettings.json` and environment variables.

### `ConsoleTest/Games/*`

Each game implements `IGame` and is responsible for its own reset, update, input handling, scoring, and title rendering. The games differ in mechanics but share the same 20x10 pixel buffer abstraction.

## PixelBoardDisplay/\*

`DisplayHelper.cs` holds board state and LCD text state. `ConsoleDisplay.cs` renders a terminal-based visualization, while `ArduinoDisplay.cs` serializes board and LCD output to hardware. `ArduinoInput.cs` maps Arduino input bytes back to keyboard events.

## PixelCatsClient/\*

The API client classes encapsulate the external service contract. `ScoreWatcher.cs` and `ProgramHelpers.cs` convert the shared JSON file into events that other tools can consume.

## Validation Logic

- The game loop in `ConsoleTest/Program.cs` validates that a current game instance exists before it is used.
- `LeaderboardClient.MintClaimCodeAsync` rejects blank game codes and signs payloads with HMAC before submission.
- `PixelCatsApiClient.SubmitCodeAsync` and `CreateScoreAndGetCodeAsync` reject blank code and game-code values.
- `ProgramHelpers.TryReadScoreWithTimestampAsync` tolerates transient I/O and JSON read failures by retrying.
- `WriteScoreFileAtomic` writes to a temporary file and then copies it into place so score consumers never need to read a half-written JSON file.
- `ConsoleTest.Tests/EducationTests.cs` and `GenericGameTests.cs` validate game state transitions, score changes, and basic invariants.

## Configuration Handling

Configuration is handled through `Microsoft.Extensions.Configuration` in `ConsoleTest/Program.cs`.

- `ConsoleTest/appsettings.json` provides defaults such as `UseEmulator` and `Leaderboard:BaseUrl`.
- Environment variables can override those values.
- Secret keys are read from `LEADERBOARD_HMAC_SNAKE`, `LEADERBOARD_HMAC_TETRIS`, and `LEADERBOARD_HMAC_EDU`.
- `PixelCatsClient/ApiClient.cs` can also inject an `API_KEY` header from the process environment.

## Error Handling

The codebase uses pragmatic error handling rather than a centralized exception pipeline.

- API calls catch network and parsing failures and return safe fallback values such as `null`, `false`, or empty arrays.
- The game loop logs failed score-code minting without crashing the main process.
- Score-file reads retry transient failures and fall back to last-write timestamps when no explicit timestamp exists.
- Hardware-related code logs serial issues instead of failing silently.

## Maintainability Considerations

- The `IGame` and `IDisplay` abstractions reduce coupling between game rules and rendering.
- The tests target contracts rather than implementation details wherever possible.
- The solution uses separate project files for console gameplay, hardware support, client utilities, and tests, which keeps dependencies explicit.
- Common data-handling helpers such as `ProgramHelpers.TryReadScoreWithTimestampAsync` reduce duplication.

## Security Considerations

- HMAC-signed claim-code requests in `LeaderboardClient` reduce the risk of unauthenticated score minting.
- API-key injection is supported in `PixelCatsApiClient`, but the repository does not contain the server-side authorization logic.
- Hardware and serial-port operations are Windows-specific and should be treated as trusted-local execution paths.
- Secrets are expected to be provided externally, which is appropriate, but the repository does not include secret management tooling.